

torch.func.jacfwd#

<https://pytorch.org/docs/stable/generated/torch.func.jacfwd.html>

Description:

Computes the Jacobian of func with respect to the arg(s) at index argnum using forward-mode autodiff

func (function) – A Python function that takes one or more arguments, one of which must be a Tensor, and returns one or more Tensors

argnums (int or Tuple[int]) – Optional, integer or tuple of integers, saying which arguments to get the Jacobian with respect to. Default: 0.

has_aux (bool) – Flag indicating that func returns a (output, aux) tuple where the first element is the output of the function to be differentiated and the second element is auxiliary objects that will not be differentiated. Default: False.

randomness (str) – Flag indicating what type of randomness to use. See vmap() for more detail. Allowed: “different”, “same”, “error”. Default: “error”

Function Signature

```
torch.func.jacfwd(func, argnums=0, has_aux=False, *,  
randomness='error')[source]#
```

Parameters

Returns

Returns:

Returns a function that takes in the same inputs as `func` and returns the Jacobian of `func` with respect to the arg(s) at `argnums`. If `has_aux` is `True`, then the returned function instead returns a `(jacobian, aux)` tuple where `jacobian` is the Jacobian and `aux` is auxiliary objects returned by `func`.

Code Examples

```
>>> from torch.func import jacfwd  
>>> x = torch.randn(5)  
>>> jacobian = jacfwd(torch.sin)(x)  
>>> expected = torch.diag(torch.cos(x))  
>>> assert torch.allclose(jacobian, expected)
```

```
>>> from torch.func import jacfwd, vmap
>>> x = torch.randn(64, 5)
>>> jacobian = vmap(jacfwd(torch.sin))(x)
>>> assert jacobian.shape == (64, 5, 5)
```

```
>>> from torch.func import jacfwd
>>> x = torch.randn(5)
>>>
>>> def f(x):
>>>     return x.sin()
>>>
>>> def g(x):
>>>     result = f(x)
>>>     return result, result
>>>
>>> jacobian_f, f_x = jacfwd(g, has_aux=True)(x)
>>> assert torch.allclose(f_x, f(x))
```

```
>>> from torch.func import jacfwd, jacrev
>>> def f(x):
>>>     return x.sin().sum()
>>>
>>> x = torch.randn(5)
>>> hessian = jacfwd(jacrev(f))(x)
>>> assert torch.allclose(hessian, torch.diag(-x.sin()))
```

```
>>> from torch.func import jacfwd
>>> def f(x, y):
>>>     return x + y ** 2
>>>
>>> x, y = torch.randn(5), torch.randn(5)
>>> jacobian = jacfwd(f, argnums=1)(x, y)
>>> expected = torch.diag(2 * y)
>>> assert torch.allclose(jacobian, expected)
```

```
>>> from torch.func import jacfwd
>>> def f(x, y):
>>>     return x + y ** 2
>>>
>>> x, y = torch.randn(5), torch.randn(5)
>>> jacobian = jacfwd(f, argnums=(0, 1))(x, y)
>>> expectedX = torch.diag(torch.ones_like(x))
>>> expectedY = torch.diag(2 * y)
>>> assert torch.allclose(jacobian[0], expectedX)
>>> assert torch.allclose(jacobian[1], expectedY)
```

Notes & Warnings

NOTE You may see this API error out with “forward-mode AD not implemented for operator X”. If so, please file a bug report and we will prioritize it. An alternative is to use `jacrev()`, which has better operator coverage.

Detailed Documentation

Documentation

Computes the Jacobian of `func` with respect to the `arg(s)` at index `argnum` using forward-mode autodiff

`func` (function) – A Python function that takes one or more arguments, one of which must be a Tensor, and returns one or more Tensors

`argnums` (int or Tuple[int]) – Optional, integer or tuple of integers, saying which arguments to get the Jacobian with respect to. Default: 0.

`has_aux` (bool) – Flag indicating that `func` returns a (output, aux) tuple where the first element is the output of the function to be differentiated and the second element is auxiliary objects that will not be differentiated. Default: False.

`randomness` (str) – Flag indicating what type of randomness to use. See `vmap()` for more detail. Allowed: “different”, “same”, “error”. Default: “error”

Returns a function that takes in the same inputs as `func` and returns the Jacobian of `func` with respect to the `arg(s)` at `argnums`. If `has_aux` is True, then the returned function instead returns a (jacobian, aux) tuple where `jacobian` is the Jacobian and `aux` is auxiliary objects returned by `func`.

You may see this API error out with “forward-mode AD not implemented for operator X”. If so, please file a bug report and we will prioritize it. An alternative is to use `jacrev()`, which has better operator coverage.

A basic usage with a pointwise, unary operation will give a diagonal array as the

Jacobian

`jacfwd()` can be composed with `vmap` to produce batched Jacobians:

If you would like to compute the output of the function as well as the jacobian of the function, use the `has_aux` flag to return the output as an auxiliary object:

Additionally, `jacrev()` can be composed with itself or `jacrev()` to produce Hessians

By default, `jacfwd()` computes the Jacobian with respect to the first input. However, it can compute the Jacobian with respect to a different argument by using `argnums`:

Additionally, passing a tuple to `argnums` will compute the Jacobian with respect to multiple arguments